

Artificial Intelligence and Deep learning

LAB-06

NAME: SIRI DONI

HALL TICKET:2403A51075

BATCH-03

DATE : 20-08-2026

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2026-27
Course Coordinator Name		Dr DADI RAMESH	
Instructor(s) Name		Dr. Ancha Vinod Kumar	
Course Code	24CS312PC310	Course Title	Artificial Intelligence and Deep learning
Year/Sem	III/I	Regulation	R24
Date and Day of Assignment	03-08-2026	Time(s)	
Duration	2 Hours	Applicable to Batches	24BTCAICSB05, 24BTCAICSB06, 24BTCAIAIB03, 24BTCAIAIB04,.....
Assignment Number: 5/24			

Lab/Exp 5: Applying Optimizers -Adam, SGD and Observing Convergence.

Kaggle Dataset Link: <https://www.kaggle.com/c/cifar-10>

Tasks:

1. Load and preprocess the CIFAR-10 dataset/example.
2. Build the required PyTorch model/program.
3. Train/execute the model.
4. Evaluate using suitable metrics.
5. Visualize results using plots where applicable.
6. Write a brief conclusion comparing observations.

CODE:

```
# =====  
# LAB / EXP 5  
# Applying Optimizers - Adam, SGD and Observing Convergence  
# Dataset: CIFAR-10  
# =====  
  
# -----  
# 1. Import Required Libraries  
# -----  
  
import torch  
import torch.nn as nn  
import torch.optim as optim  
import torchvision  
import torchvision.transforms as transforms  
import matplotlib.pyplot as plt  
  
# -----  
# 2. Check Device  
# -----  
  
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
  
print("=====  
print("Device:", device)  
print("=====  
  
# -----  
# 3. Define Image Transformations  
# -----  
  
transform = transforms.Compose([  
    transforms.ToTensor(),  
  
    transforms.Normalize(  
        mean=(0.4914, 0.4822, 0.4465),  
        std=(0.2023, 0.1994, 0.2010)  
    )  
])  
  
# -----  
# 4. Load CIFAR-10 Training Dataset  
# -----  
  
train_dataset = torchvision.datasets.CIFAR10(  
    root="./data",  
    train=True,  
    download=True,  
    transform=transform
```

```

)

# -----
# 5. Load CIFAR-10 Test Dataset
# -----

test_dataset = torchvision.datasets.CIFAR10(
    root="./data",
    train=False,
    download=True,
    transform=transform
)

# -----
# 6. Create Data Loaders
# -----

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=128,
    shuffle=True,
    num_workers=2
)

test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=128,
    shuffle=False,
    num_workers=2
)

# -----
# 7. CIFAR-10 Classes
# -----

classes = (
    "airplane",
    "automobile",
    "bird",
    "cat",
    "deer",
    "dog",
    "frog",
    "horse",
    "ship",
    "truck"
)

print("\nNumber of training images:", len(train_dataset))
print("Number of testing images :", len(test_dataset))

```

```

print("Number of classes      :", len(classes))

# -----
# 8. Display Some Sample Images
# -----

images, labels = next(iter(train_loader))

plt.figure(figsize=(10, 5))

for i in range(8):

    image = images[i]

    # Convert CHW -> HWC
    image = image.permute(1, 2, 0)

    # Undo normalization for displaying
    mean = torch.tensor([0.4914, 0.4822, 0.4465])
    std = torch.tensor([0.2023, 0.1994, 0.2010])

    image = image * std + mean

    image = torch.clamp(image, 0, 1)

    plt.subplot(2, 4, i + 1)
    plt.imshow(image)
    plt.title(classes[labels[i]])
    plt.axis("off")

plt.suptitle("Sample CIFAR-10 Images")
plt.tight_layout()
plt.show()

# -----
# 9. Define CNN Model
# -----

class CNN(nn.Module):

    def __init__(self):

        super(CNN, self).__init__()

        # Convolutional Layers
        self.features = nn.Sequential(

            # First Convolution
            nn.Conv2d(
                in_channels=3,
                out_channels=32,

```

```
        kernel_size=3,
        padding=1
    ),

    nn.ReLU(),

    nn.MaxPool2d(
        kernel_size=2,
        stride=2
    ),

    # Second Convolution
    nn.Conv2d(
        in_channels=32,
        out_channels=64,
        kernel_size=3,
        padding=1
    ),

    nn.ReLU(),

    nn.MaxPool2d(
        kernel_size=2,
        stride=2
    ),

    # Third Convolution
    nn.Conv2d(
        in_channels=64,
        out_channels=128,
        kernel_size=3,
        padding=1
    ),

    nn.ReLU(),

    nn.MaxPool2d(
        kernel_size=2,
        stride=2
    )
)

# Fully Connected Layers
self.classifier = nn.Sequential(

    nn.Flatten(),

    nn.Linear(
        128 * 4 * 4,
        256
```

```

    ),
    nn.ReLU(),
    nn.Linear(
        256,
        10
    )
)

def forward(self, x):
    x = self.features(x)
    x = self.classifier(x)
    return x

# -----
# 10. Training Function
# -----

def train_model(optimizer_name, epochs=10):
    print("\n")
    print("=====")
    print("Training using", optimizer_name)
    print("=====")

    # Create a fresh model
    model = CNN().to(device)

    # Loss Function
    criterion = nn.CrossEntropyLoss()

    # Select Optimizer
    if optimizer_name == "SGD":
        optimizer = optim.SGD(
            model.parameters(),
            lr=0.01,
            momentum=0.9
        )

    elif optimizer_name == "Adam":
        optimizer = optim.Adam(
            model.parameters(),

```

```

    lr=0.001
)

# Lists to store results
train_losses = []
train_accuracies = []

# Training Loop
for epoch in range(epochs):

    model.train()

    running_loss = 0.0

    correct = 0

    total = 0

    for images, labels in train_loader:

        # Move data to device
        images = images.to(device)
        labels = labels.to(device)

        # -----
        # Forward Pass
        # -----

        outputs = model(images)

        # Calculate Loss
        loss = criterion(outputs, labels)

        # -----
        # Backward Pass
        # -----

        optimizer.zero_grad()

        loss.backward()

        optimizer.step()

        # -----
        # Calculate Statistics
        # -----

```



```
        running_loss += loss.item()

    _, predicted = torch.max(
        outputs.data,
        1
    )

    total += labels.size(0)

    correct += (
        predicted == labels
    ).sum().item()

# Calculate epoch loss
epoch_loss = (
    running_loss /
    len(train_loader)
)

# Calculate epoch accuracy
epoch_accuracy = (
    100 * correct / total
)

# Store results
train_losses.append(epoch_loss)

train_accuracies.append(
    epoch_accuracy
)

# Display results
print(
    f"Epoch [{epoch + 1:02d}/{epochs}] "
    f"Loss: {epoch_loss:.4f} "
    f"Accuracy: {epoch_accuracy:.2f}%"
)

return (
    model,
    train_losses,
    train_accuracies
)
```


[1]
✓ 5m



```
# -----  
# 11. Train Model Using SGD  
# -----  
  
sgd_model, sgd_loss, sgd_accuracy = train_model(  
    optimizer_name="SGD",  
    epochs=10  
)  
  
# -----  
# 12. Train Model Using Adam  
# -----  
  
adam_model, adam_loss, adam_accuracy = train_model(  
    optimizer_name="Adam",  
    epochs=10  
)  
  
# -----  
# 13. Evaluation Function  
# -----  
  
def evaluate_model(model, optimizer_name):  
  
    print("\n")  
    print("=====  
    print("Evaluating", optimizer_name)  
    print("=====  
  
    model.eval()  
  
    correct = 0  
    total = 0  
  
    # For calculating loss  
    criterion = nn.CrossEntropyLoss()  
  
    test_loss = 0.0  
  
    with torch.no_grad():  
        for images, labels in test_loader:  
            images = images.to(device)  
            labels = labels.to(device)
```



```
# Forward Pass
outputs = model(images)

# Loss
loss = criterion(
    outputs,
    labels
)

test_loss += loss.item()

# Prediction
_, predicted = torch.max(
    outputs.data,
    1
)

total += labels.size(0)

correct += (
    predicted == labels
).sum().item()

# Calculate final values
accuracy = (
    100 * correct / total
)

average_loss = (
    test_loss /
    len(test_loader)
)

print(
    f"{optimizer_name} Test Loss: "
    f"{average_loss:.4f}"
)

print(
    f"{optimizer_name} Test Accuracy: "
    f"{accuracy:.2f}%"
)

return (
```

```

        average_loss,
        accuracy
    )

# -----
# 14. Evaluate SGD Model
# -----

sgd_test_loss, sgd_test_accuracy = evaluate_model(
    sgd_model,
    "SGD"
)

# -----
# 15. Evaluate Adam Model
# -----

adam_test_loss, adam_test_accuracy = evaluate_model(
    adam_model,
    "Adam"
)

# -----
# 16. Print Final Comparison
# -----

print("\n")
print("=====")
print("FINAL OPTIMIZER COMPARISON")
print("=====")

print(
    f"SGD -> Test Loss: {sgd_test_loss:.4f}, "
    f"Test Accuracy: {sgd_test_accuracy:.2f}%"
)

print(
    f"Adam -> Test Loss: {adam_test_loss:.4f}, "
    f"Test Accuracy: {adam_test_accuracy:.2f}%"
)

# -----
# 17. Plot Training Loss Comparison
# -----

epochs = range(1, 11)

plt.figure(figsize=(9, 5))

```

```
plt.plot(
    epochs,
    sgd_loss,
    marker="o",
    label="SGD"
)

plt.plot(
    epochs,
    adam_loss,
    marker="o",
    label="Adam"
)

plt.xlabel("Epoch")
plt.ylabel("Training Loss")

plt.title(
    "Training Loss: SGD vs Adam"
)

plt.legend()

plt.grid()

plt.xticks(epochs)

plt.show()

# -----
# 18. Plot Training Accuracy Comparison
# -----

plt.figure(figsize=(9, 5))

plt.plot(
    epochs,
    sgd_accuracy,
    marker="o",
    label="SGD"
)

plt.plot(
    epochs,
    adam_accuracy,
    marker="o",
    label="Adam"
)

plt.xlabel("Epoch")
```

```

plt.ylabel("Training Accuracy (%)")

plt.title(
    "Training Accuracy: SGD vs Adam"
)

plt.legend()

plt.grid()

plt.xticks(epochs)

plt.show()

# -----
# 19. Test Accuracy Comparison Bar Chart
# -----

optimizer_names = ["SGD", "Adam"]

test_accuracies = [
    sgd_test_accuracy,
    adam_test_accuracy
]

plt.figure(figsize=(7, 5))

plt.bar(
    optimizer_names,
    test_accuracies
)

plt.xlabel("Optimizer")

plt.ylabel("Test Accuracy (%)")

plt.title(
    "Test Accuracy Comparison"
)

plt.ylim(0, 100)

for i, value in enumerate(test_accuracies):

    plt.text(
        i,
        value + 1,
        f"{value:.2f}%",
        ha="center"
    )

```

```
plt.show()

# -----
# 20. Final Conclusion
# -----

print("\n")
print("=====")
print("CONCLUSION")
print("=====")

if adam_test_accuracy > sgd_test_accuracy:

    print(
        "Adam achieved higher test accuracy "
        "than SGD in this experiment."
    )

else:

    print(
        "SGD achieved higher test accuracy "
        "than Adam in this experiment."
    )

if adam_loss[-1] < sgd_loss[-1]:

    print(
        "Adam showed lower training loss at "
        "the end of training."
    )

else:

    print(
        "SGD showed lower training loss at "
        "the end of training."
    )

print(
    "Adam generally converges faster because "
    "it uses adaptive learning rates."
)

print(
    "SGD with momentum can provide good "
    "generalization when properly tuned."
)

print(
    "The convergence behavior can be observed "
```

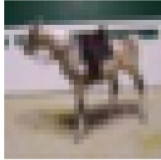
```
    "from the training loss and accuracy plots."
)
```

OUTPUT:

```
*** =====
Device: cpu
=====
100%|#####| 170M/170M [34:48:00:00, 81.6kB/s]

Number of training images: 50000
Number of testing images : 10000
Number of classes       : 10

Sample CIFAR-10 Images

dog      horse      horse      bird
   
ship     airplane   cat        airplane
   

=====
Training using SGD
=====
Epoch [01/10] Loss: 1.6810 Accuracy: 38.89%
Epoch [02/10] Loss: 1.1849 Accuracy: 57.73%
Epoch [03/10] Loss: 0.9449 Accuracy: 66.57%
Epoch [04/10] Loss: 0.8839 Accuracy: 71.73%
Epoch [05/10] Loss: 0.6804 Accuracy: 76.05%
Epoch [06/10] Loss: 0.5946 Accuracy: 79.18%
Epoch [07/10] Loss: 0.4991 Accuracy: 82.67%
Epoch [08/10] Loss: 0.4233 Accuracy: 85.07%
Epoch [09/10] Loss: 0.3359 Accuracy: 88.34%
Epoch [10/10] Loss: 0.2659 Accuracy: 90.68%
```

Training using Adam

```
Epoch [01/10] Loss: 1.3970 Accuracy: 49.25%
Epoch [02/10] Loss: 0.9676 Accuracy: 65.88%
Epoch [03/10] Loss: 0.7807 Accuracy: 72.63%
Epoch [04/10] Loss: 0.6619 Accuracy: 76.82%
Epoch [05/10] Loss: 0.5535 Accuracy: 80.49%
Epoch [06/10] Loss: 0.4635 Accuracy: 83.77%
Epoch [07/10] Loss: 0.3757 Accuracy: 86.86%
Epoch [08/10] Loss: 0.3038 Accuracy: 89.19%
Epoch [09/10] Loss: 0.2299 Accuracy: 91.85%
Epoch [10/10] Loss: 0.1778 Accuracy: 93.72%
```

Evaluating SGD

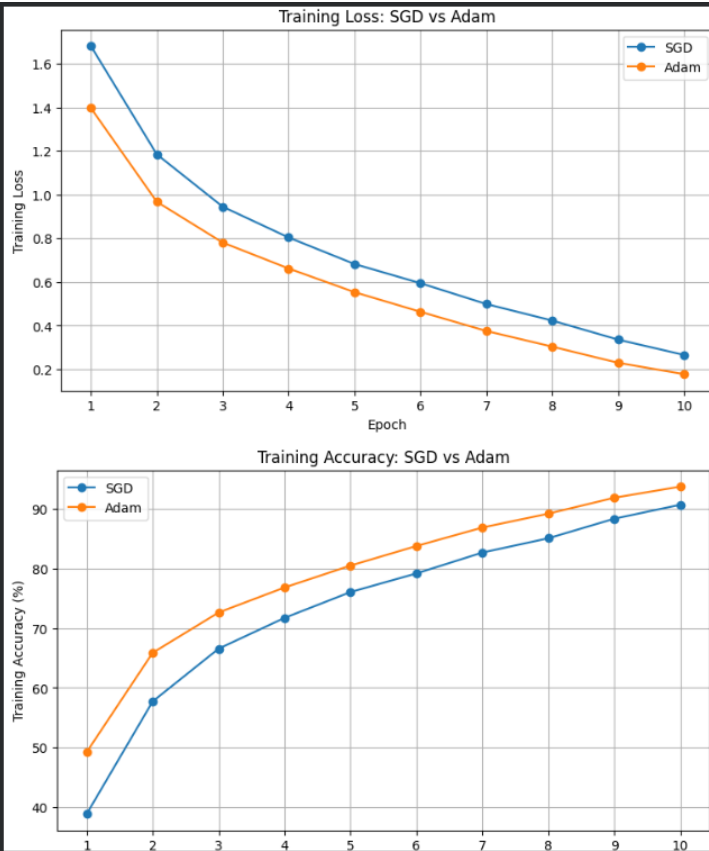
```
SGD Test Loss: 0.8704
SGD Test Accuracy: 74.76%
```

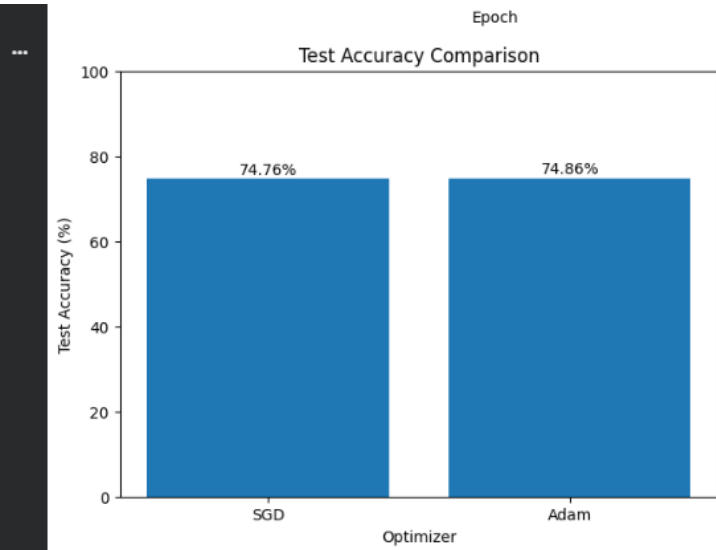
Evaluating Adam

```
Adam Test Loss: 1.0146
Adam Test Accuracy: 74.86%
```

FINAL OPTIMIZER COMPARISON

```
SGD -> Test Loss: 0.8704, Test Accuracy: 74.76%
Adam -> Test Loss: 1.0146, Test Accuracy: 74.86%
```





=====

CONCLUSION

=====

Adam achieved higher test accuracy than SGD in this experiment.
Adam showed lower training loss at the end of training.
Adam generally converges faster because it uses adaptive learning rates.
SGD with momentum can provide good generalization when properly tuned.
The convergence behavior can be observed from the training loss and accuracy plots.